CS 671 — Deep Generative Models

Variational Autoencoders

ELBO Derivation — Complete Proof with Intuition, Examples & PyTorch Implementation

Dr. Aditya Nigam | Department of Computer Science & Engineering, IIT Mandi

Indian Institute of Technology Mandi

Motivation · Two Derivations · **Reparameterisation** · PyTorch · **Applications**

Lecture Roadmap — 8 Sections

01 Motivation & Big Picture

Why generative models? Real-world applications.

02 The Generative Model

Setup, notation, prior, likelihood.

03 Variational Inference

Approximate posterior, KL divergence.

04 ELBO — Two Full Derivations

KL decomposition + Jensen's inequality.

05 The Two ELBO Terms

Reconstruction + KL regularisation.

06 Reparameterisation Trick

Sampling, gradients & backprop fix.

07 Training & Extensions

Algorithm, β-VAE, modern variants.

08 Summary & Mind Maps

Key takeaways & concept overview.

01 SECTION

Motivation & Big Picture

Why do we need generative models?

In this section:

- ▶ 10 real-world application domains
- ▶ GAN vs VAE vs Flow vs Diffusion
- ▶ VAEs: principled + interpretable

VAE Powers Real-World Applications Across Every Domain

Image Generation	Drug Discovery	Audio Synthesis	Text Generation	Genomics
StyleGAN · DALL-E Stable Diffusion	Novel molecule design	Voice cloning Music generation	Story writing Code completion	Protein structure prediction

Anomaly Detection	Representation Learning	Image Inpainting	Neuroscience	Medical Imaging
Fraud & fault detection	Disentangled features	Restoration & completion	Brain signal modelling	Tumour & disease detection

What is a Generative Model?

> ***"If you can generate something, you truly understand it."*** — Feynman principle (adapted)

GAN	VAE	Flow	Diff.
Adversarial Generator vs D	Variational Bayes (Today!)	Normalising Flows	Diffusion Models (DALL-E)
High quality but unstable	**Principled, smooth latent**	**Exact density invertible**	**SOTA quality slow sample**
	TODAY'S FOCUS		

VAEs combine **deep learning** with **Bayesian inference** — principled, interpretable, continuous generation

02 SECTION

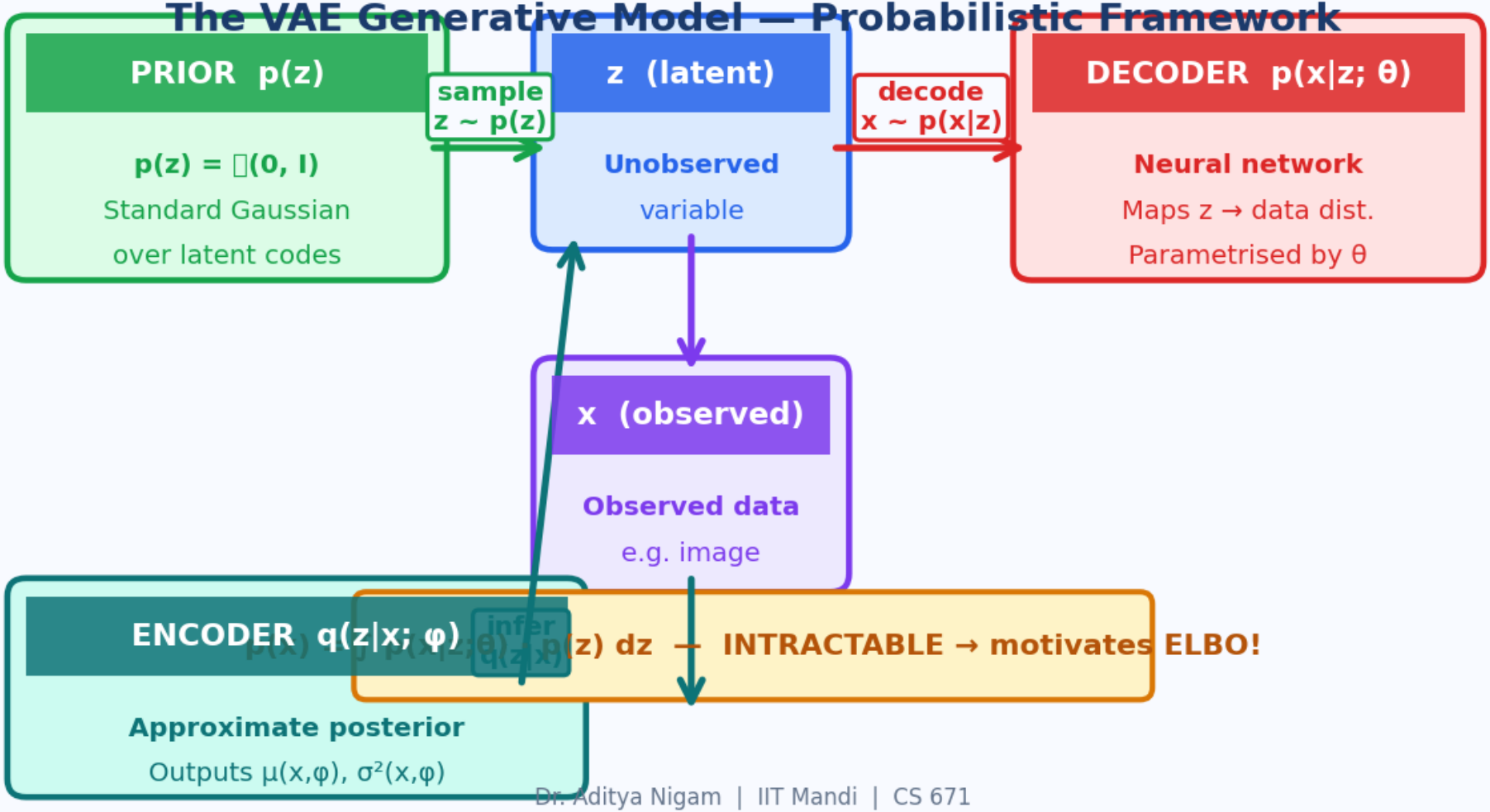
The Generative Model

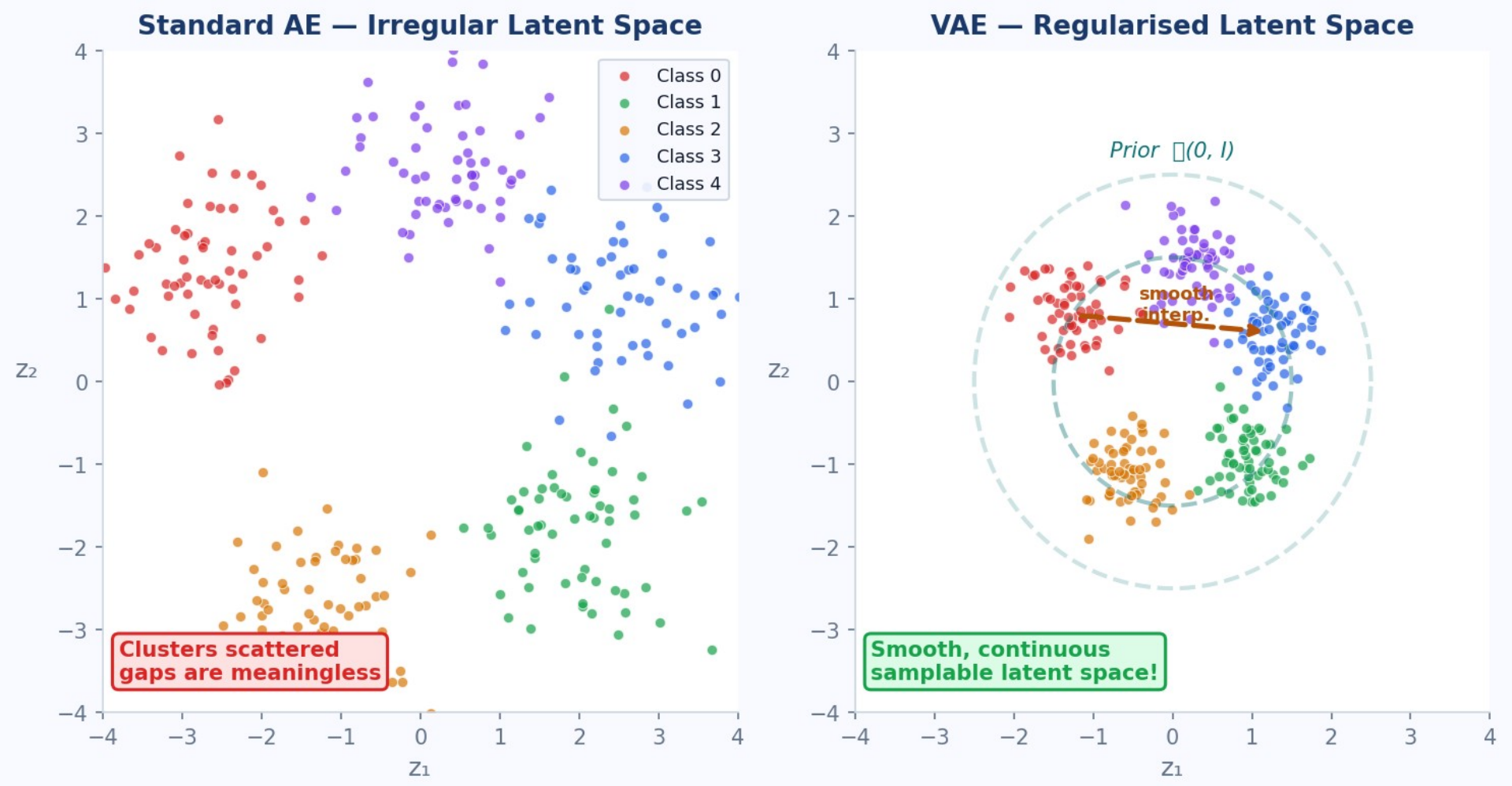
Probabilistic setup: prior, likelihood, posterior

In this section:

z	Latent variable z and prior $p(z) = \mathcal{N}(0, I)$
D	Decoder: $p(x \mid z; \theta)$ — neural network
!	Why $p(x) = \int p(x \mid z)p(z)dz$ is intractable

The VAE Generative Model — Probabilistic Framework




Standard AE — Irregular Latent Space
Class 0
Class 1
Class 2
Class 3
Class 4
Clusters scattered
gaps are meaningless
z₂
z₁
VAE — Regularised Latent Space
Prior ℕ(0, I)
smooth
interp.
Smooth, continuous
samplable latent space!
z₂
z₁

Why Direct Maximisation of log p(x) is Intractable

What We WANT

$$\max \quad \log p(x;\theta)$$

$$= \log \int p(x|z;\theta)\, p(z)\, dz$$

- ✓ Principled MLE objective
- ✓ Directly trains p(x)
- ✗ Integrates over ALL of z
- ✗ No closed form (neural decoder)
- ✗ Circular: p(z|x) needs p(x)

✗ INTRACTABLE

The ELBO Solution

$$\max \quad \text{ELBO}(\theta,\varphi;x)$$

$$\log p(x;\theta) \geq \text{ELBO}$$

- ✓ Tractable lower bound
- ✓ No intractable integral
- ✓ Closed-form KL Gaussian
- ✓ MC estimate for recon.
- ✓ Fully differentiable

ELBO is tight when $q(z|x;\varphi) = p(z|x;\theta)$ → maximising ELBO ≡ minimising KL gap

03 SECTION

Variational Inference

Replacing the intractable posterior with a tractable family

In this section:

q	Introduce $q_\varphi(z\|x) = \mathcal{N}(\mu(x,\varphi), \sigma^2(x,\varphi))$
KL	$KL[q\|\|p] \geq 0$ measures approximation quality
NN	Amortised encoder neural network

Variational Inference — Core Idea

Problem: $p(z|x;\theta) = p(x|z;\theta)\cdot p(z) / p(x;\theta)$ is INTRACTABLE — computing $p(x;\theta)$ requires the integral we cannot solve.

Solution: introduce a tractable approximate posterior $q_\varphi(z|x)$

Encoder: $q_\varphi(z \mid x) = \mathcal{N}(z;\ \mu(x,\varphi),\ \mathrm{diag}(\sigma^2(x,\varphi)))$

Diagonal Gaussian

Fully factorised — simple, differentiable, closed-form KL

Neural Network

$\mu(x,\varphi)$ and $\sigma^2(x,\varphi)$ are NN outputs — universal approximators

Amortised

One network for ALL inputs x — scales to large datasets

KL Divergence

$KL[q_\varphi(z|x) \| p(z|x;\theta)] \geq 0$ quantifies approximation error

04 SECTION

ELBO — Two Full Derivations

KL Decomposition and Jensen's Inequality

In this section:

1	Method 1: KL decomposition → isolate log p(x)
2	Method 2: Importance weighting + Jensen
=	log p(x) = KL[q‖p(z|x)] + ELBO

ELBO Derivation — Five Steps

1 $\log p(x;\theta) = \log \int p(x,z;\theta)\, dz$ — *Marginalise joint over z*

2 $= \log \int q(z|x) \cdot [p(x,z;\theta)/q(z|x)]\, dz$ — *Multiply & divide by q(z|x)*

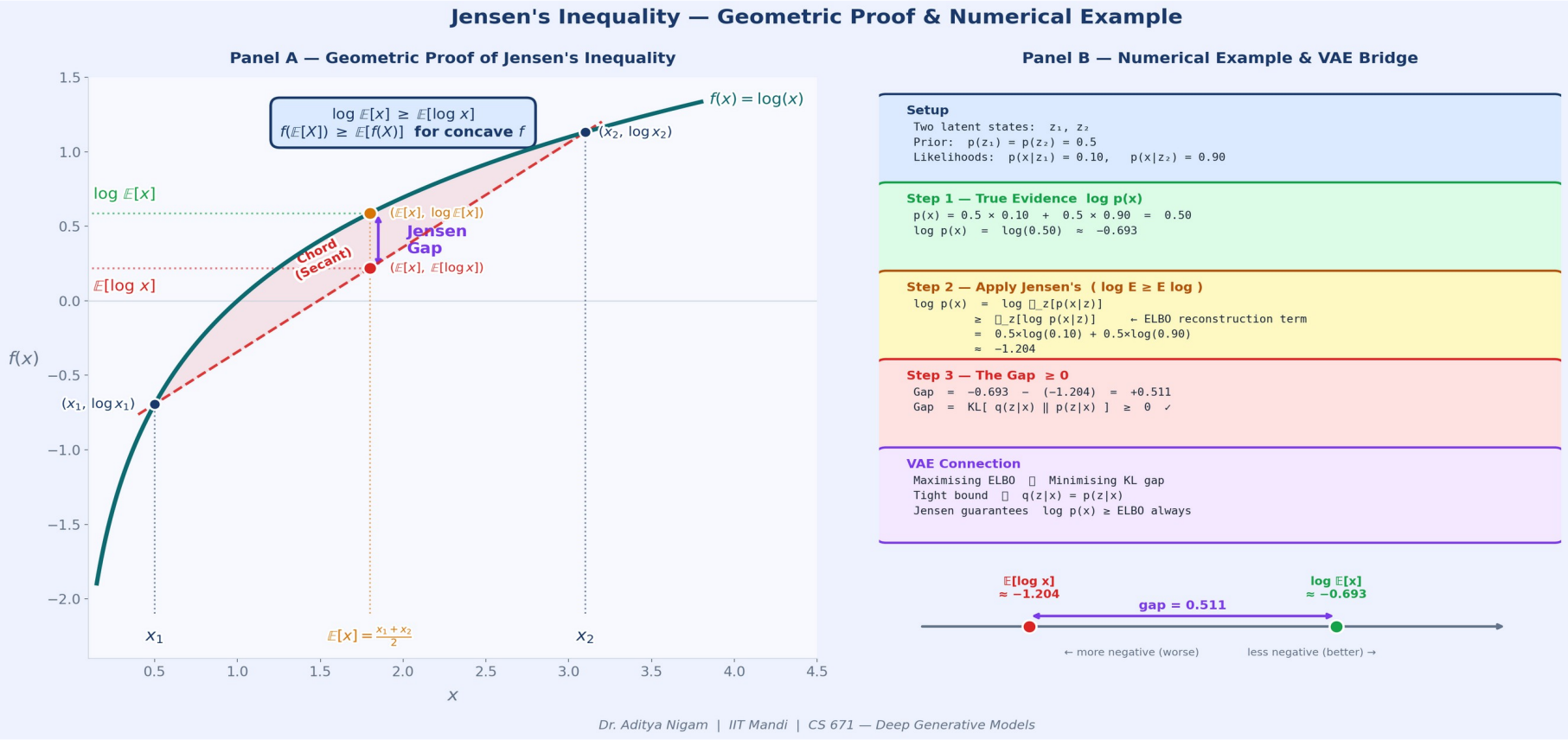
3 $= \log \mathbb{E}_{q(z|x)}[\, p(x,z;\theta) \,/\, q(z|x)\,]$ — *Recognise expectation under q*

4 $\geq \mathbb{E}_{q(z|x)}[\, \log p(x,z;\theta) - \log q(z|x)\,]$ — *Jensen's ineq. — log is concave*

5 $= \mathbb{E}_q[\log p(x|z;\theta)] - KL[\, q(z|x) \,\|\, p(z)\,]$ — *Expand joint, group KL → ELBO!*

$$ELBO(\theta,\varphi;x) = \mathbb{E}_q[\log p(x|z;\theta)] - KL[\, q_\varphi(z|x) \,\|\, p(z)\,]$$

Method 2: Jensen's Inequality — The Key Step



The Fundamental ELBO Identity

$$\log p(x;\theta) = KL[\, q_\varphi(z|x) \,\|\, p(z|x;\theta)\,] + ELBO(\theta,\varphi;x)$$

Since KL ≥ 0 always: $\log p(x;\theta) \geq ELBO(\theta,\varphi;x)$ — the lower bound!

Maximise ELBO

Pushes log p(x) upward — increasing data likelihood

Gap = KL[q‖p(z|x)]

Closes only when encoder = true posterior exactly

Reconstruction term

$\mathbb{E}_{q}[\log p(x|z)]$ — decoder quality under encoder samples

KL regularisation

KL[q(z|x)‖p(z)] — encoder stays close to prior $\mathcal{N}(0,I)$

Tight bound

ELBO = log p(x) iff q(z|x;φ) = p(z|x;θ) exactly

Amortised VI

One encoder network — scales efficiently to large data

05 SECTION

The Two ELBO Terms

Reconstruction loss and KL regularisation in depth

In this section:

R	Reconstruction: $\mathbb{E}_q[\log p(x\|z)]$ — fidelity
K	KL: closed-form for Gaussians — no sampling
B	Balance: reconstruction vs regularisation

① Reconstruction Term

$$\mathbb{E}_{q(z|x;\varphi)}\left[\log p(x|z;\theta)\right]$$

Input x Recon $\hat{x}$

Measures reconstruction fidelity

Binary data → Cross-entropy loss

Continuous data → MSE loss

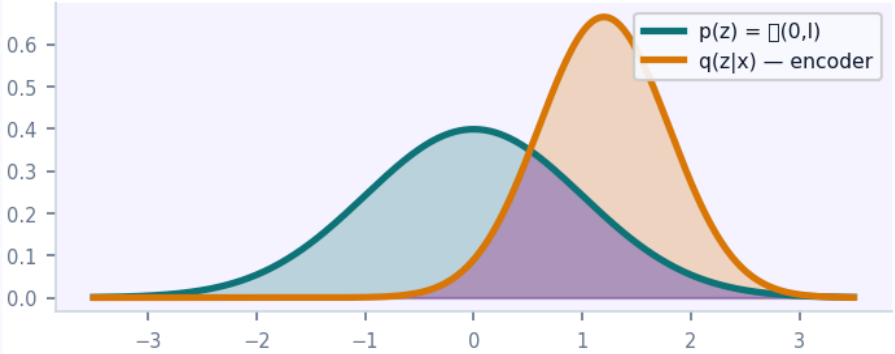
↑ Maximise → Better decoder

"How well can we reconstruct the input?"

② KL Regularisation

$$KL\left[q(z|x;\varphi) \,\|\, p(z)\right]$$

KL gap = shaded area → minimise



Measures distance from prior 𝒩(0,I)

Closed form for Gaussians:

$$\tfrac{1}{2}\sum_j(\mu_j^2 + \sigma_j^2 - \log\sigma_j^2 - 1)$$

↓ Minimise → Structured latent space

"How regular is the latent space?"

Closed-Form KL Divergence for Gaussians

When $q_\varphi(z|x) = \mathcal{N}(\mu, \sigma^2 \mathbf{I})$ and $p(z) = \mathcal{N}(0, \mathbf{I})$ — KL has an elegant closed form:

$$\mathrm{KL}[\, q_\varphi(z|x) \,\|\, p(z) \,] = \tfrac{1}{2} \cdot \Sigma_j \,[\, \mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1 \,]$$

$\mathrm{KL}[q\|p] = \mathbb{E}_q[\, \log q(z) - \log p(z) \,]$ — *Definition of KL divergence*

$= \mathbb{E}_q[\, -\tfrac{1}{2}\, \Sigma(z-\mu)^2/\sigma^2 + \tfrac{1}{2}\, \Sigma z^2 \,] + \text{const}$ — *Expand log-densities of Gaussians*

$= \tfrac{1}{2} \cdot \Sigma_j \,[\, \mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1 \,]$ ✓ — *Compute expectations analytically*

No sampling needed for KL — computed analytically from encoder outputs μ and σ only!

06 SECTION

The Reparameterisation Trick

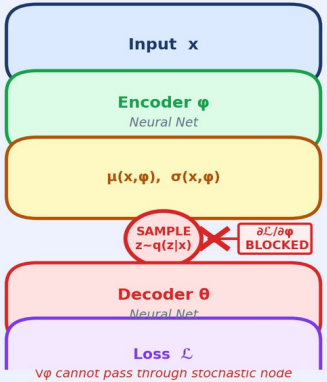
Making sampling differentiable for backpropagation

In this section:

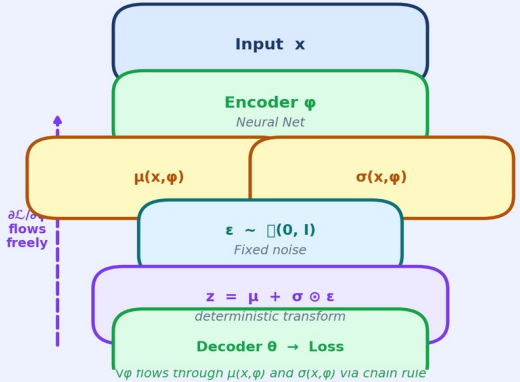
× Sampling $z \sim q$ breaks gradient flow through φ	= $z = \mu_\varphi(x) + \sigma_\varphi(x) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$	✓ Chain rule now gives $\partial \mathcal{L} / \partial \varphi$ cleanly

The Reparameterisation Trick — Why It Works & How to Implement It

Naïve Sampling (Gradient Cannot Flow ✗)



Reparameterisation Trick (Gradient Flows Freely ✓)



Mathematical Equivalence

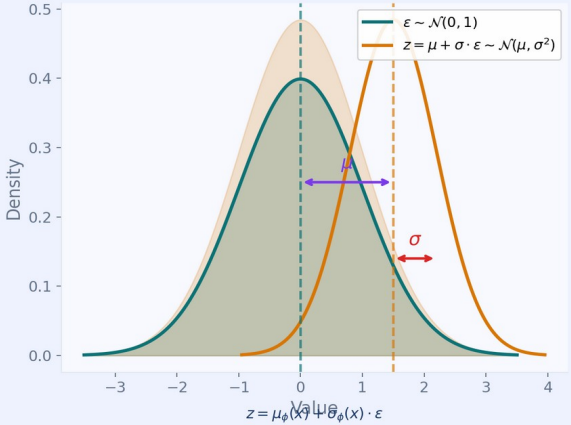
```
# Before — Naïve Sampling
z ~ q_phi(z|x)
  = N(mu_phi(x), sigma^2_phi(x))

d/d_phi  E_q[f(z)]
  = ???  (no gradient!)
```

```
# After — Reparameterisation
eps ~ N(0, I)   [fixed noise]
z   = mu + sigma * eps

d/d_phi  E_eps[f(z(eps,phi))]
  = E_eps[df/dz * dz/d_phi] ✓
```

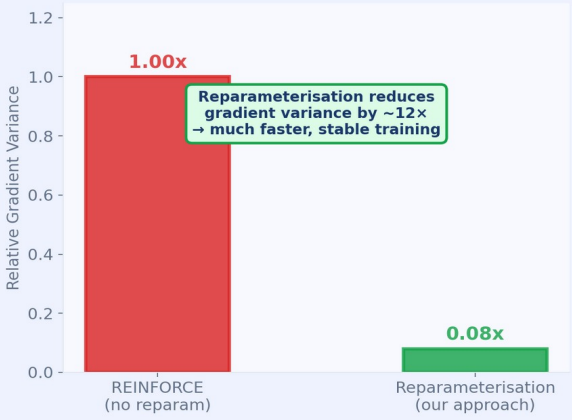
Gaussian Reparameterisation: Shift & Scale of Standard Normal



$z = \mu_\phi(x) + \sigma_\phi(x) \cdot \varepsilon$

moves stochasticity to ε — outside the gradient path

Why It Matters: Variance of Gradient Estimators



PyTorch Implementation (3 Lines)

```
# Encoder outputs
mu, log_var = encoder(x)

# Reparameterisation
std = torch.exp(0.5 * log_var)
eps = torch.randn_like(std)   # ε~N(0,I)
z   = mu + std * eps          # z=μ+σ⊙ε

# Forward pass
x_recon = decoder(z)

# ELBO loss
recon_loss = F.binary_cross_entropy(
    x_recon, x, reduction="sum")
kl_loss = -0.5 * torch.sum(
    1 + log_var - mu**2 - log_var.exp())
loss = recon_loss + kl_loss
loss.backward()  # ✓ gradients flow!
```

→ chain rule applies → ∂loss/∂φ is well-defined

Reparameterisation — Formula & PyTorch

The Trick — Step by Step

Step 1 — Sample fixed noise:

$$\varepsilon \sim \mathcal{N}(0, \mathrm{I})$$

Step 2 — Deterministic transform:

$$z = \mu_\varphi(x) + \sigma_\varphi(x) \odot \varepsilon$$

Result:

$$z \sim \mathcal{N}(\mu_\varphi(x), \sigma^2_\varphi(x) \cdot \mathrm{I}) \quad \checkmark$$

Gradient:

$$\partial z / \partial \varphi = \partial \mu / \partial \varphi + \varepsilon \cdot \partial \sigma / \partial \varphi$$

φ appears only inside μ() and σ()
→ chain rule gives $\partial \mathcal{L} / \partial \varphi$ cleanly

PyTorch Implementation

```
# Encoder forward pass
mu, log_var = encoder(x)

# Reparameterise
std = torch.exp(0.5 * log_var)
eps = torch.randn_like(std)
z   = mu  +  std * eps

# Decoder & Losses
x_hat = decoder(z)
recon = F.binary_cross_entropy(
    x_hat, x, reduction='sum')
kl = -0.5 * torch.sum(
  1+log_var-mu**2-log_var.exp())
loss = recon + kl
loss.backward()  # gradients flow!
```

07 SECTION

Training, Results & Extensions

Complete algorithm, β-VAE, and modern variants

In this section:

6	6-step training algorithm with reparameterisation
β	β-VAE: disentangled latent dimensions
★	VQ-VAE, WAE, NVAE, LDM (Stable Diffusion)

Complete VAE Training Algorithm

1 **Sample mini-batch**

$\{x^{(1)},\ldots,x^{(M)}\}$ from dataset $\mathcal{D}$

2 **Encode**

$\mu^{(i)},\ \sigma^{2(i)} \ = \ \text{encoder\_}\varphi(\ x^{(i)}\ )$

3 **Reparameterise**

$z^{(i)} \ = \ \mu^{(i)} \ + \ \sigma^{(i)} \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0,I)$

4 **Decode**

$\hat{x}^{(i)} \ = \ \text{decoder\_}\theta(\ z^{(i)}\ )$

5 **Compute Loss**

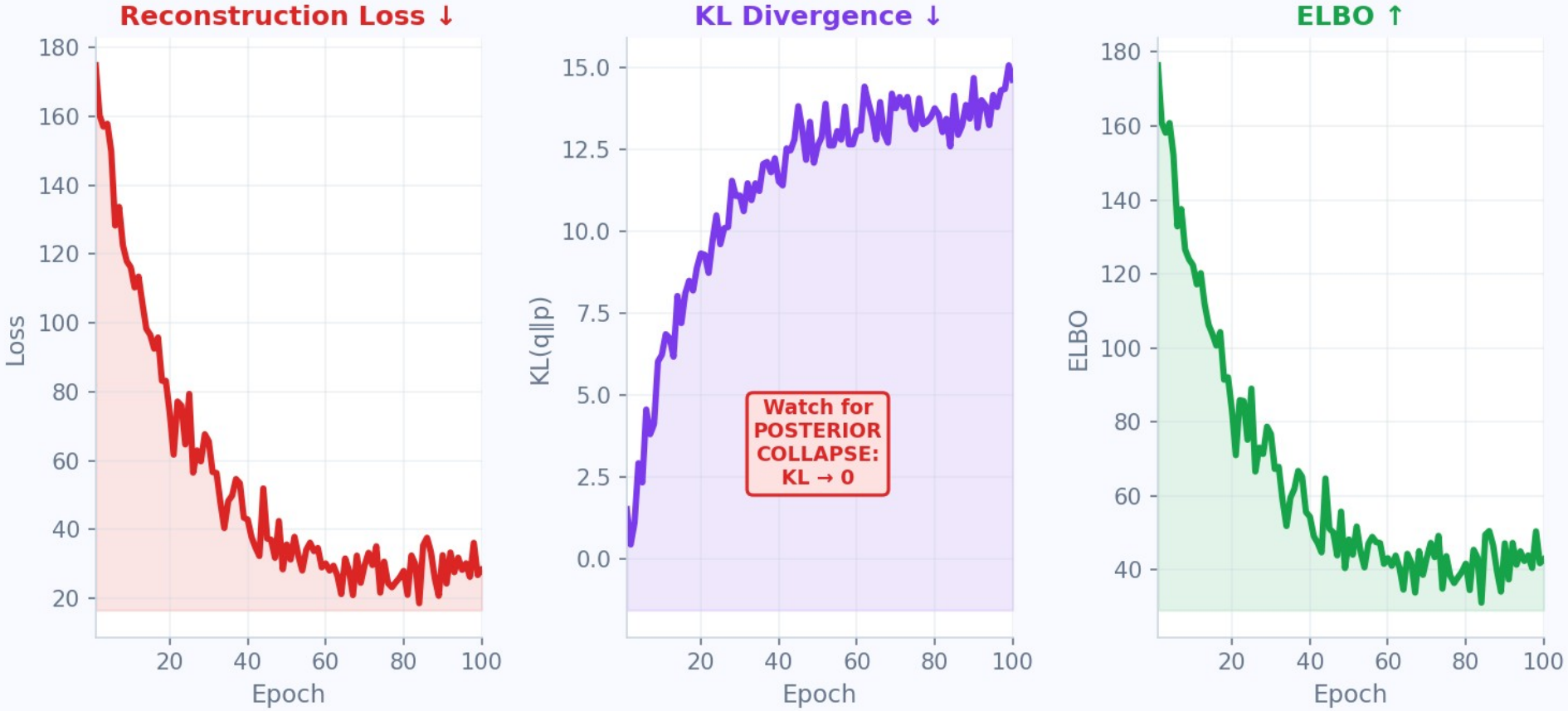
$\mathcal{L} \ = \ -\mathbb{E}\text{\_}q[\log\ p(x|z;\theta)] \ + \ \mathrm{KL}[\ q \parallel p\ ]$

6 **Update Parameters**

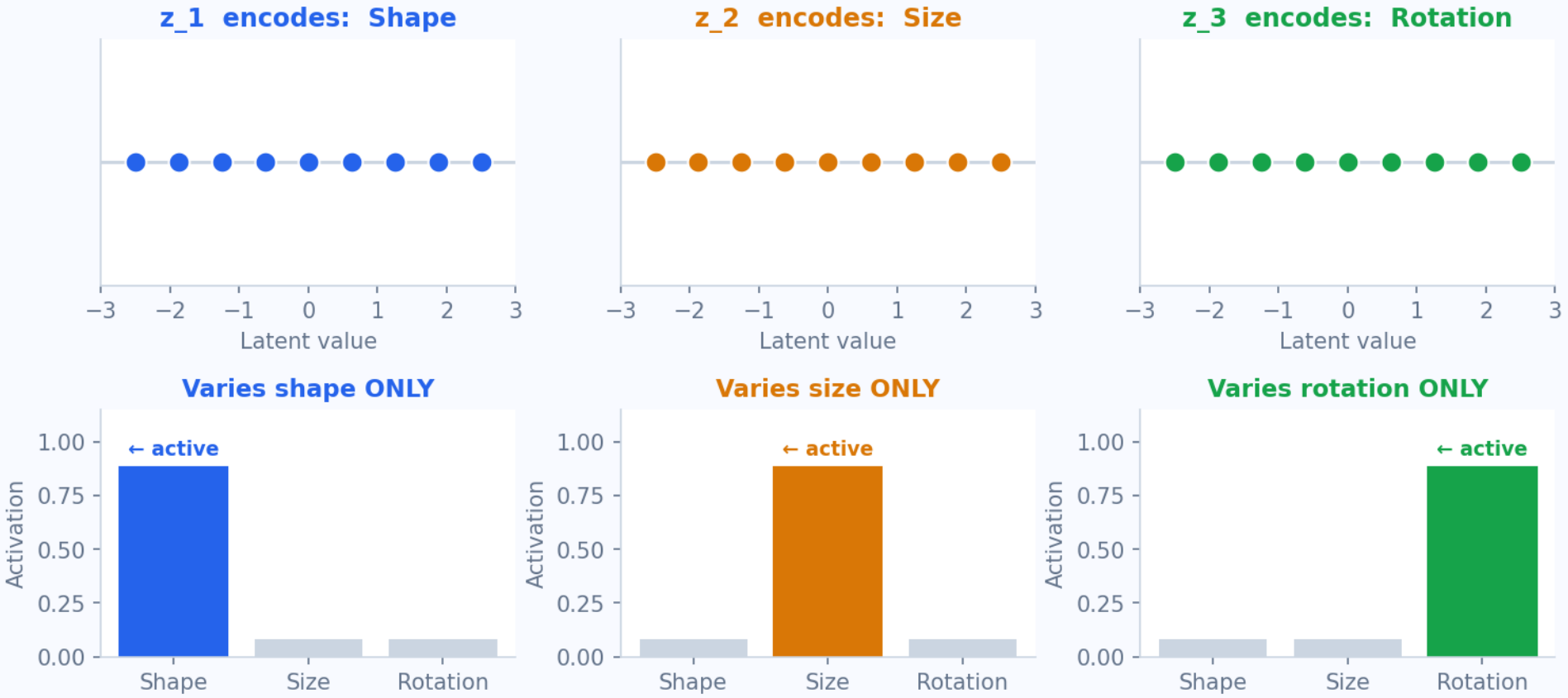
$\theta \leftarrow \theta - \eta\nabla\text{\_}\theta\mathcal{L} \qquad \varphi \leftarrow \varphi - \eta\nabla\text{\_}\varphi\mathcal{L} \qquad (\text{Adam})$

$\mathcal{L}(\theta,\varphi;x) \ = \ \text{Reconstruction Loss} \ + \ \text{KL Penalty} \qquad [\text{minimise with gradient descent}]$

VAE Training Dynamics — Three Key Metrics



β-VAE: Disentangled Latent Dimensions (β > 1 enforces independence)



Modern VAE Variants — Building on ELBO

VAE (2013)

Kingma & Welling. Gaussian prior + diagonal posterior. ELBO = Recon + KL[q‖$\mathcal{N}$(0,I)].

β-VAE (2017)

Higgins et al. β·KL term (β>1) enforces disentangled latents. Trade-off: reconstruction vs interpretability.

VQ-VAE (2017)

van den Oord et al. Discrete codebook replaces Gaussian latent. Foundation of DALL-E-1, music generation.

WAE (2018)

Tolstikhin et al. Wasserstein distance replaces KL divergence. Better quality, less posterior collapse.

NVAE (2020)

Vahdat & Kautz. Deep hierarchical VAE with residual cells. SOTA on high-resolution generation.

LDM (2022)

Rombach et al. VAE encoder + Diffusion model. Foundation of Stable Diffusion, DALL-E-3.

08 SECTION

Summary & Mind Maps

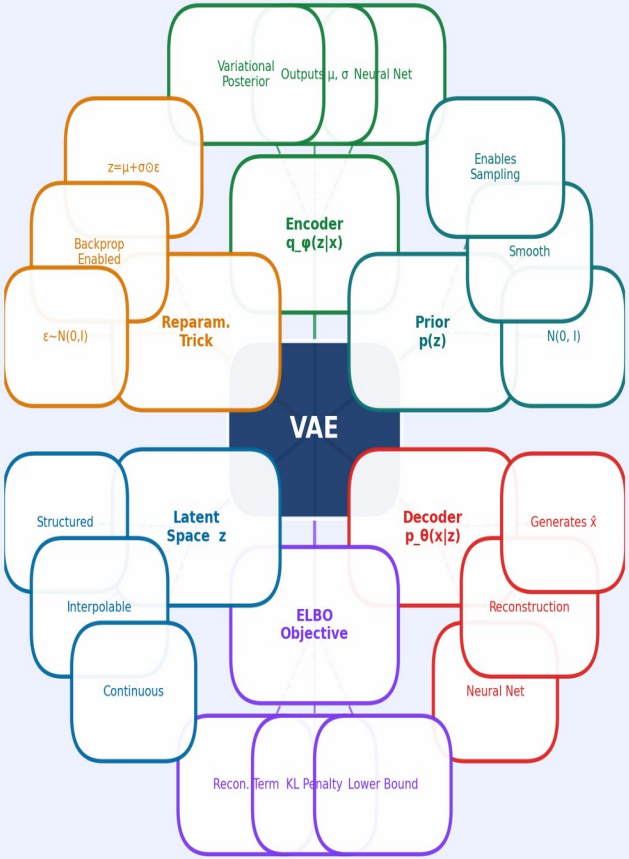
Everything consolidated on two slides

In this section:

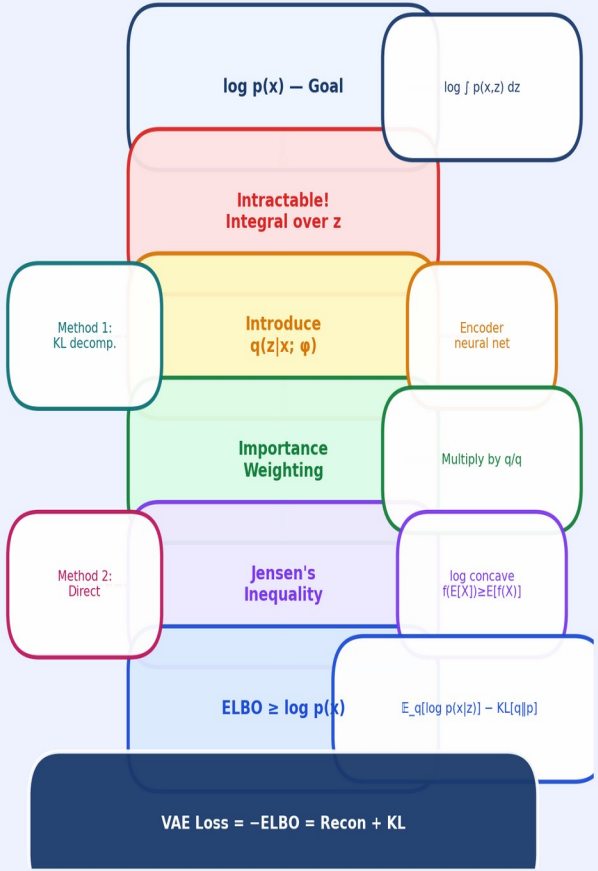
M	3 concept mind maps — components, derivation, reparam
E	5 key equations at a glance
T	5 core takeaways for exam preparation

VAE Concept Mind Maps

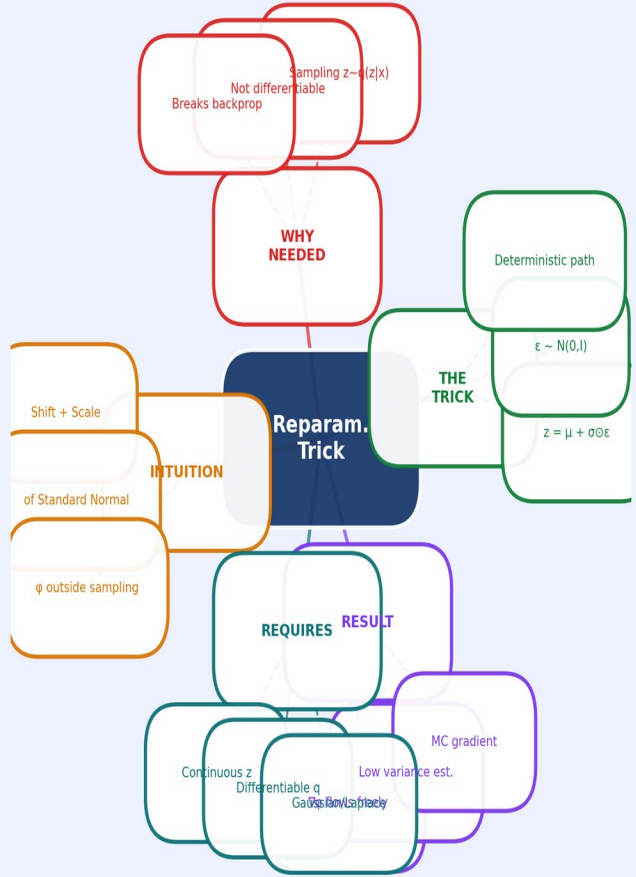
Mind Map 1 — VAE Core Components



Mind Map 2 — ELBO Derivation Roadmap



Mind Map 3 — Reparameterisation Trick



Key Results at a Glance

ELBO Identity

$$\log p(x;\theta) = KL[\, q_\varphi(z|x) \,\|\, p(z|x;\theta) \,] + ELBO(\theta,\varphi;x)$$

Fundamental decomposition — derived by both methods

ELBO

$$\mathbb{E}_q[\, \log p(x|z;\theta) \,] - KL[\, q_\varphi(z|x) \,\|\, p(z) \,]$$

Reconstruction quality minus KL regularisation

Closed-form KL

$$\tfrac{1}{2} \cdot \Sigma_j \, [\, \mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1 \,]$$

Analytic — no sampling needed for regularisation term

Reparameterisation

$$z = \mu_\varphi(x) + \sigma_\varphi(x) \odot \varepsilon \,, \qquad \varepsilon \sim \mathcal{N}(0, I)$$

Moves stochasticity outside gradient computation path

VAE Loss Function

$$\mathcal{L}(\theta,\varphi;x) = \text{Reconstruction Loss} + \text{KL Penalty}$$

Minimised via gradient descent — Adam optimiser

Thank You!

Questions? Let's discuss!

5 Core Takeaways

- VAEs are principled generative models combining deep learning with Bayesian variational inference.
- ELBO = lower bound on log p(x) — derived via KL decomposition OR Jensen's inequality.
- Two terms: Reconstruction fidelity + KL regularisation pushing encoder toward prior $\mathcal{N}(0,I)$.
- Reparameterisation trick moves sampling outside the gradient path — enabling backprop through z.
- Modern variants (β-VAE, VQ-VAE, LDM / Stable Diffusion) build directly on this ELBO foundation.